Exercise 1: The "Base Page" & Chaining (Navigation) The Task: Create a `BasePage` that contains common methods (like `click`, `find`, and `get_text`). Then, create a `HomePage` class. What to test: Navigate to the "Laptops" category from the sidebar. POM Logic: The `click_laptops()` method in `HomePage` should return a new instance of the `LaptopsCategoryPage`. The Goal: Your test should look like this: `home.click_laptops().verify_laptop_list_presence()`.

Project Structure

```
project/
|
 ├── pages/
|    ├── base_page.py
|    ├── home_page.py
|    └── laptops_category_page.py
|
 ├── tests/
|    └── test_navigation.py
|
 ├── conftest.py
└── requirements.txt
```

base_page.py

```
from selenium.webdriver.common.by import By


class BasePage:

    def __init__(self, driver):
        self.driver = driver
```

```
    def click(self, locator):
        self.driver.find_element(*locator).click()

    def find(self, locator):
        return self.driver.find_element(*locator)

    def get_text(self, locator):
        return self.driver.find_element(*locator).text
```

home_page.py

```
from selenium.webdriver.common.by import By
from pages.base_page import BasePage
from pages.laptops_category_page import LaptopsCategoryPage


class HomePage(BasePage):

    LAPTOPS_LINK = (By.LINK_TEXT, "Laptops")

    def click_laptops(self):
        self.click(self.LAPTOPS_LINK)

        # Return next page object
        return LaptopsCategoryPage(self.driver)
```

laptops_category_page.py

```
from selenium.webdriver.common.by import By
from pages.base_page import BasePage
```

```
class LaptopsCategoryPage(BasePage):
    PRODUCTS = (By.CLASS_NAME, "card-title")
    def verify_laptop_list_presence(self):
        products = self.driver.find_elements(*self.PRODUCTS)
        assert len(products) > 0, "Laptop products are not displayed"
        print("Laptop list is displayed successfully")
```

test_navigation.py

```
from pages.home_page import HomePage

def test_laptop_navigation(driver):
    home = HomePage(driver)
    home.click_laptops().verify_laptop_list_presence()
```

conftest.py

```
import pytest
from selenium import webdriver
```

```
@pytest.fixture
def driver():

    driver = webdriver.Chrome()

    driver.maximize_window()

    driver.get("https://www.demoblaze.com/index.html")

    yield driver

    driver.quit()
```

requirements.txt

```
selenium
pytest
```

Exercise 2: List Elements & Dynamic Data (Product Search) The Task: Create a `CategoryPage` that handles the list of products displayed. What to test: Navigate to the "Phones" category. Use `find_elements` to capture all product names displayed on the page. POM Logic: Write a method `get_all_product_names()` that returns a list of strings. The Goal: In your test, assert that "Samsung galaxy s6" is present in the returned list.

 # base_page.py

```
class BasePage:
```

```
    def __init__(self, driver):
        self.driver = driver

    def click(self, locator):
        self.driver.find_element(*locator).click()

    def find(self, locator):
        return self.driver.find_element(*locator)

    def get_text(self, locator):
        return self.driver.find_element(*locator).text
```

 # category_page.py

```
from selenium.webdriver.common.by import By
from pages.base_page import BasePage


class CategoryPage(BasePage):

    PHONES_LINK = (By.LINK_TEXT, "Phones")
    PRODUCTS = (By.CLASS_NAME, "card-title")

    def click_phones(self):
        self.click(self.PHONES_LINK)
```

```
    def get_all_product_names(self):

        product_elements = self.driver.find_elements(*self.PRODUCTS)

        product_names = []

        for product in product_elements:
            product_names.append(product.text)

        return product_names
```

test_products.py

```
from pages.category_page import CategoryPage

def test_phone_products(driver):
    page = CategoryPage(driver)
    page.click_phones()
    products = page.get_all_product_names()
    assert "Samsung galaxy s6" in products
```

conftest.py

```
import pytest
from selenium import webdriver

@pytest.fixture
def driver():

    driver = webdriver.Chrome()
    driver.maximize_window()
    driver.get("https://www.demoblaze.com/index.html")
    yield driver
    driver.quit()
```

Exercise 3: Synchronization & Alerts (Adding to Cart) The Task: Create a `ProductDetailsPage`. What to test: Click on "Sony vaio i5", then click the "Add to cart" button. POM Logic: When "Add to cart" is clicked, a browser alert (JavaScript pop-up) appears after a short delay. The Goal: Inside your POM method `add_product_to_cart()`, implement a `WebDriverWait` for `alert_is_present()`, accept the alert, and log the action using the `logging` module.

base_page.py

```
class BasePage:

    def __init__(self, driver):
```

```
        self.driver = driver

    def click(self, locator):
        self.driver.find_element(*locator).click()

    def find(self, locator):
        return self.driver.find_element(*locator)

    def get_text(self, locator):
        return self.driver.find_element(*locator).text
```

product_details_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from pages.base_page import BasePage
import logging

logging.basicConfig(level=logging.INFO)

class ProductDetailsPage(BasePage):

    SONY_VAIO_I5 = (By.LINK_TEXT, "Sony vaio i5")
    ADD_TO_CART = (By.LINK_TEXT, "Add to cart")
```

```
    def open_product(self):
        self.click(self.SONY_VAIO_I5)

    def add_product_to_cart(self):

        self.click(self.ADD_TO_CART)

        WebDriverWait(self.driver, 10).until(
            EC.alert_is_present()
        )

        alert = self.driver.switch_to.alert

        alert.accept()

        logging.info("Product added to cart successfully")
```

test_add_to_cart.py

```
from pages.product_details_page import ProductDetailsPage

def test_add_product_to_cart(driver):
    product = ProductDetailsPage(driver)
    product.open_product()
```

```
    product.add_product_to_cart()
```

conftest.py

```
import pytest
from selenium import webdriver

@pytest.fixture
def driver():

    driver = webdriver.Chrome()
    driver.maximize_window()
    driver.get("https://www.demoblaze.com/index.html")
    yield driver
    driver.quit()
```

Exercise 4: Multi-Step Flow & Component POM (Checkout) The Task: Create a `CartPage` and a `PurchaseModal` (as a component or separate class). What to test: Navigate to the "Cart", click "Place Order", fill in the modal form (Name, Country, City, Card, Month, Year), and click "Purchase". POM Logic: Encapsulate the form fields within a `fill_purchase_form(data_dict)` method that takes a dictionary of info. The Goal: Verify the "Thank you for your purchase!" success message appears. This test must trigger an Allure Step for every field filled.

base_page.py

```
class BasePage:
```

```
    def __init__(self, driver):
        self.driver = driver

    def click(self, locator):
        self.driver.find_element(*locator).click()

    def find(self, locator):
        return self.driver.find_element(*locator)

    def enter_text(self, locator, text):
        self.driver.find_element(*locator).send_keys(text)

    def get_text(self, locator):
        return self.driver.find_element(*locator).text
```

purchase_modal.py

```
from selenium.webdriver.common.by import By
from pages.base_page import BasePage
import allure


class PurchaseModal(BasePage):

    NAME = (By.ID, "name")
    COUNTRY = (By.ID, "country")
```

```
CITY = (By.ID, "city")
CARD = (By.ID, "card")
MONTH = (By.ID, "month")
YEAR = (By.ID, "year")
PURCHASE_BUTTON = (By.XPATH, "//button[text()='Purchase']")

def fill_purchase_form(self, data_dict):

    with allure.step("Enter Name"):
        self.enter_text(self.NAME, data_dict["name"])

    with allure.step("Enter Country"):
        self.enter_text(self.COUNTRY, data_dict["country"])

    with allure.step("Enter City"):
        self.enter_text(self.CITY, data_dict["city"])

    with allure.step("Enter Card"):
        self.enter_text(self.CARD, data_dict["card"])

    with allure.step("Enter Month"):
        self.enter_text(self.MONTH, data_dict["month"])

    with allure.step("Enter Year"):
        self.enter_text(self.YEAR, data_dict["year"])
```

```
    def click_purchase(self):
        self.click(self.PURCHASE_BUTTON)
```

 # cart_page.py

```
from selenium.webdriver.common.by import By
from pages.base_page import BasePage
from pages.purchase_modal import PurchaseModal


class CartPage(BasePage):

    CART_LINK = (By.ID, "cartur")
    PLACE_ORDER = (By.XPATH, "//button[text()='Place Order']")
    SUCCESS_MESSAGE = (By.XPATH, "//h2[text()='Thank you for your purchase!']")

    def open_cart(self):
        self.click(self.CART_LINK)

    def click_place_order(self):
        self.click(self.PLACE_ORDER)
        return PurchaseModal(self.driver)

    def verify_success_message(self):
        success_text = self.get_text(self.SUCCESS_MESSAGE)
        assert "Thank you for your purchase!" in success_text
```

test_checkout.py

```
from pages.cart_page import CartPage

def test_checkout(driver):
    cart = CartPage(driver)
    cart.open_cart()
    purchase = cart.click_place_order()
    data = {
        "name": "Ashish",
        "country": "India",
        "city": "Delhi",
        "card": "123456789",
        "month": "May",
        "year": "2026"
    }
    purchase.fill_purchase_form(data)
    purchase.click_purchase()
    cart.verify_success_message()
```

conftest.py

```
import pytest
from selenium import webdriver
```

```
@pytest.fixture
def driver():
    driver = webdriver.Chrome()
    driver.maximize_window()
    driver.get("https://www.demoblaze.com/index.html")
    yield driver
    driver.quit()
```

Exercise 5: Error Handling & Screenshots (Negative Test) The Task: Handle the "Login" modal. What to test: Click "Log in" in the header, enter a valid username but a wrong password, and click "Log in". POM Logic: Use `pytest` hooks in `conftest.py` to monitor this test. The Goal: Since the login fails, an alert saying "Wrong password." appears. Your test should catch this. Screenshot Challenge: Intentionally write an assertion that fails (e.g., asserting the alert text is "Success"). The `conftest.py` must automatically capture a screenshot of the login modal and attach it to the Allure Report.

 # base_page.py

```
class BasePage:

    def __init__(self, driver):
        self.driver = driver

    def click(self, locator):
        self.driver.find_element(*locator).click()
```

```
    def enter_text(self, locator, text):
        self.driver.find_element(*locator).send_keys(text)

    def get_text(self, locator):
        return self.driver.find_element(*locator).text
```

login_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from pages.base_page import BasePage


class LoginPage(BasePage):

    LOGIN_HEADER = (By.ID, "login2")
    USERNAME = (By.ID, "loginusername")
    PASSWORD = (By.ID, "loginpassword")
    LOGIN_BUTTON = (By.XPATH, "//button[text()='Log in']")

    def open_login_modal(self):
        self.click(self.LOGIN_HEADER)

    def login(self, username, password):
        self.enter_text(self.USERNAME, username)
```

```
        self.enter_text(self.PASSWORD, password)
        self.click(self.LOGIN_BUTTON)

    def get_alert_text(self):

        WebDriverWait(self.driver, 10).until(
            EC.alert_is_present()
        )
        alert = self.driver.switch_to.alert
        alert_text = alert.text
        alert.accept()
        return alert_text
```

test_login_negative.py

```
from pages.login_page import LoginPage

def test_invalid_login(driver):
    login = LoginPage(driver)
    login.open_login_modal()
    login.login("testuser", "wrongpassword")
    alert_text = login.get_alert_text()
    assert alert_text == "Success"
```

conftest.py

```
import pytest
import allure
from selenium import webdriver

@pytest.fixture
def driver():

    driver = webdriver.Chrome()
    driver.maximize_window()

    driver.get("https://www.demoblaze.com/index.html")
    yield driver
    driver.quit()
@pytest.hookimpl(hookwrapper=True)

def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()

    if report.when == "call" and report.failed:
        driver = item.funcargs["driver"]
        screenshot = driver.get_screenshot_as_png()
        allure.attach(
            screenshot,
```

```
        name="Failure Screenshot",
        attachment_type=allure.attachment_type.PNG
    )
```